

Customer Behavior Prediction

Machine Learning Engineering Project - Final Report

An end-to-end customer analytics system covering campaign response prediction, customer segmentation, historical customer value estimation, explainable AI, deep learning experiments, API deployment, dashboard visualization, PostgreSQL persistence, testing, and containerized deployment.

Primary Classification Model	LightGBM
Customer Segmentation	K-Means
Customer Value Model	Random Forest Regression
Explainability	SHAP + LIME + PDP
Application Layer	FastAPI + Streamlit
Persistence	PostgreSQL
Deployment	Docker Compose
Automated Tests	26 passing
Test Coverage	76%

1. Executive Summary

This project implements an end-to-end machine learning system for analysing customer behaviour and supporting marketing decisions. The solution combines supervised classification, regression, unsupervised customer segmentation, explainable AI, deep learning experiments, API deployment, an interactive dashboard, database persistence, automated testing, and containerized deployment.

The primary campaign-response model is LightGBM. The deployed classification threshold is 0.20, selected to improve recall for likely responders while maintaining useful precision. Customer segmentation is performed using K-Means with two business-readable customer groups. Customer value is estimated using Random Forest regression.

The customer-value target used in this project is a historical spending proxy derived from Total Spending. It must therefore not be interpreted as a forecast of true future customer lifetime value. This limitation is retained explicitly in both the application dashboard and this report.

Explainability is provided using SHAP for feature-level model contributions, LIME for local rule-based explanations, and partial dependence analysis for selected important features. The deep-learning experiments include an artificial neural network and an autoencoder for anomaly detection. An LSTM was not fabricated because the available dataset does not contain the transaction-level time-ordered purchase-event sequences required for a valid sequence model.

The final application stack consists of FastAPI, Streamlit, PostgreSQL, and Docker Compose. The complete Dockerized stack was validated successfully. Automated testing currently contains 26 passing tests with 76% source coverage, including explicit data-leakage prevention tests. Ruff static analysis also passes without errors.

2. Dataset and Data Preparation

The project uses the marketing campaign customer dataset stored under data/raw/marketing_campaign.csv. The raw dataset contains 2,240 customer records and 29 columns covering demographic information, household characteristics, purchasing behaviour, campaign interactions, and campaign response.

Data quality checks included duplicate inspection, missing-value analysis, type inspection, and age-outlier detection. There were 24 missing Income values, which were replaced using the median Income. No duplicate rows were detected. Three unrealistic age records above 100 years were removed. The resulting validated dataset contains 2,237 customer records.

The campaign Response target is imbalanced: approximately 85.07% of customers are non-responders and 14.93% are responders. This imbalance was considered when interpreting precision, recall, F1 score, ROC-AUC, and classification thresholds rather than relying on accuracy alone.

3. Feature Engineering

Feature	Definition
Total_Spending	Sum of spending across wines, fruits, meat, fish, sweet products, and gold products.
Total_Children	Kidhome + Teenhome.
Age	Reference year 2026 - Year_Birth.
Customer_Tenure	Reference year 2026 - year of Dt_Customer.
Categorical Encoding	Education and Marital_Status encoded using one-hot encoding with drop_first=True.

Customer ID, the original customer-enrolment date, and birth year are removed from the final modelling inputs after the required engineered features are created. The final response classification model uses 38 input features.

The production data pipeline is implemented under `src/data` and `src/features`. Running `python -m src.data.pipeline` performs raw data loading, cleaning, feature engineering, and processed-data generation without requiring manual notebook execution. The verified processed output contains 2,237 rows, 39 columns including the target, and no missing values.

Explicit leakage-prevention tests verify that Response is not included among model inputs, customer ID is excluded, and raw `Dt_Customer` and `Year_Birth` fields are removed after their derived features are created.

4. Classical Machine Learning Experiments

Multiple classification algorithms were evaluated for campaign response prediction. The comparison considered not only accuracy but also precision, recall, F1 score, and ROC-AUC because of the class imbalance in the Response target.

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression	87.95%	65.12%	41.79%	50.91%	88.04%
Decision Tree	88.17%	73.33%	32.84%	45.36%	72.91%
Random Forest	89.06%	82.14%	34.33%	48.42%	90.35%
KNN	87.95%	65.85%	40.30%	50.00%	78.07%
SVM	88.62%	75.00%	35.82%	48.48%	88.43%
LightGBM (0.50)	89.06%	70.45%	46.27%	55.86%	91.04%
LightGBM (0.20)	87.95%	60.00%	58.21%	59.09%	91.04%

LightGBM provided the strongest ROC-AUC among the evaluated classical models at approximately 0.9104. At the default 0.50 classification threshold, recall was 46.27%. Threshold analysis was therefore performed to improve responder detection.

The deployed threshold is 0.20. At this threshold, the model achieved 87.95% accuracy, 60.00% precision, 58.21% recall, 59.09% F1 score, and approximately 91.04% ROC-AUC. The lower threshold intentionally trades some precision for improved recall, allowing more potential campaign responders to be identified.

During experimentation, an identifier-leakage issue involving customer ID was detected and corrected before final model selection. A validation-splitting issue in an intermediate XGBoost experiment was also identified and was not used as the basis for the deployed model. These corrections are important for preserving trustworthy evaluation.

5. Final Response Prediction Model

The final response-prediction artifact is the LightGBM model stored in `models/final_lightgbm_model.pkl` together with its configuration in `models/model_config.pkl`. The configuration preserves the exact 38-feature input order and the deployed classification threshold of 0.20.

The production predictor validates that all expected features are present, reorders incoming data to the saved feature order, generates response probabilities, applies the configured threshold, and returns both the numeric prediction and the business-readable labels 'Likely to Respond' or 'Unlikely to Respond'.

6. Customer Segmentation

Customer segmentation was performed to identify groups with different behavioural and commercial characteristics. The segmentation features include Income, Total Spending, Total Children, Age, Recency, and the numbers of Web, Catalog, and Store purchases.

K-Means was selected as the primary clustering approach after evaluating cluster quality using the Silhouette Score and Davies-Bouldin Index. A two-cluster solution provided a clear and business-interpretable separation of the customer base.

Method	Clusters	Silhouette	Davies-Bouldin
K-Means	2	0.3275	1.2728
Gaussian Mixture Model	2	0.3091	1.3110

6.1 Segment Profiles

Measure	Cluster 0	Cluster 1
Customers	1,226	1,011
Average Income	37,339.85	70,280.97
Average Total Spending	146.30	1,162.89
Average Children	1.26	0.57
Average Age	55.30	59.28
Average Recency	48.88	49.38
Average Web Purchases	2.59	5.90
Average Catalog Purchases	0.74	4.99
Average Store Purchases	3.60	8.45
Campaign Response Rate	9.62%	21.37%

Cluster 1 represents the higher-value customer group. These customers have substantially higher average income and spending and make more purchases through web, catalog, and store channels. Cluster 0 represents the comparatively lower-value customer group.

A standardized value score based on spending, web purchases, catalog purchases, store purchases, and customer tenure produced an average score of -0.543 for Cluster 0 and 0.658 for Cluster 1. Income was deliberately excluded from this derived value score because the dataset contains an extreme Income outlier.

The observed campaign response rate was approximately 9.62% for Cluster 0 and 21.37% for Cluster 1. Therefore, the higher-value cluster showed roughly 2.22 times the observed response rate of the lower-value cluster. This is an association in the available data and should not be interpreted as evidence that segment membership causes campaign response.

Gaussian Mixture Models were also evaluated as an alternative clustering technique. For two clusters, GMM achieved a Silhouette Score of 0.3091 and a Davies-Bouldin Index of 1.3110, compared with 0.3275 and 1.2728 respectively for K-Means. K-Means was therefore retained as the primary segmentation model because it produced slightly stronger separation on both metrics and straightforward business interpretation.

7. Customer Value Estimation

The project also investigates customer value using regression. The available dataset does not contain future customer spending or a true lifetime-value label. Therefore, Total Spending is used as a historical customer-value proxy, named CLV_Proxy. This distinction is important: the model estimates historical value from the available customer behaviour and must not be interpreted as a true forecast of future Customer Lifetime Value.

To prevent target leakage, CLV_Proxy itself is excluded from the predictor variables. The regression model uses Income, NumWebPurchases, NumCatalogPurchases, NumStorePurchases, and Customer_Tenure as its five input

features.

Model	MAE	RMSE	R-Squared
Linear Regression	198.28	298.69	0.7651
Random Forest Regression	109.57	208.75	0.8853

Random Forest Regression clearly outperformed the linear baseline and was selected as the final customer-value model. On the held-out test set it achieved a Mean Absolute Error of approximately 109.57, Root Mean Squared Error of 208.75, and R-Squared of 0.8853.

7.1 Random Forest Feature Importance

Feature	Importance
NumCatalogPurchases	0.6252
Income	0.1956
NumStorePurchases	0.1189
NumWebPurchases	0.0416
Customer_Tenure	0.0187

NumCatalogPurchases was the dominant feature in the fitted Random Forest regression model, followed by Income and NumStorePurchases. These importance values describe how the trained model uses the available predictors; they should not be interpreted as causal effects on customer spending.

The production artifact is stored as `models/final_clv_rf_model.pkl` with its corresponding model configuration. Predictions exposed through the application are labelled as `Predicted_CLV_Proxy` so that the historical-proxy limitation remains visible to users.

8. Model Explainability

Explainability was incorporated so that response predictions can be interpreted at both global and individual-customer levels. The project uses SHAP, LIME, and Partial Dependence Plots (PDP) as complementary explanation techniques.

8.1 SHAP Analysis

SHAP was used to explain how individual features contribute to the response model output. Global SHAP analysis identifies the features that most strongly influence predictions across the evaluated customers, while local SHAP explanations show how specific feature values push an individual prediction toward or away from the positive response class.

Representative Case	Response Probability	Interpretation
Low probability	0.000000038	Very unlikely to respond
Borderline	0.20046	Close to deployed 0.20 threshold
High probability	0.99925	Very likely to respond

Representative low-, high-, and borderline-probability customers were selected for individual explanation. The borderline example is particularly useful because its probability of approximately 0.20046 lies very close to the deployed decision threshold of 0.20.

8.2 LIME Analysis

LIME was implemented as a second local explanation method. For an individual customer, it constructs a local approximation around that prediction and reports feature rules that increase or decrease support for the positive response class. The production dashboard displays the top LIME contributions alongside SHAP explanations.

SHAP and LIME should be interpreted as complementary rather than numerically interchangeable methods. SHAP values and LIME contribution weights are produced using different explanation frameworks, so their magnitudes should not be compared directly. Agreement or disagreement in feature direction can instead be used as an additional qualitative diagnostic.

8.3 Partial Dependence Analysis

Partial Dependence Plots were examined for selected response-model features. The fitted model showed a generally negative relationship between Recency and predicted response, a nonlinear relationship for Total Spending, and an increase in predicted response across NumWebVisitsMonth up to approximately nine visits followed by a relative plateau.

These patterns describe behaviour learned by the fitted model within this dataset. They are not causal conclusions and should not be interpreted as evidence that directly changing one of these customer attributes would necessarily cause a corresponding change in campaign response.

8.4 Explainability in the Application

Reusable explanation modules are implemented under `src/explainability`. The Streamlit dashboard presents the five most influential SHAP features and the five most influential LIME rules for an entered customer profile, allowing prediction results to be accompanied by human-readable model reasoning.

9. Deep Learning Experiments

Deep learning experiments were included for comparison and learning purposes. The available tabular dataset is relatively small, so the production response model remains the boosted-tree LightGBM model rather than a neural network.

9.1 Artificial Neural Network

The Artificial Neural Network used standardized versions of the same 38 response-prediction features. The data was separated into training, validation, and untouched test partitions. Feature scaling was fitted only on the training data to prevent leakage.

The network architecture consisted of dense layers with 64, 32, and 16 hidden units, ReLU activation, batch normalization, and dropout with a rate of 0.30. The output layer used a sigmoid activation for binary classification. Early stopping was used during training, with the best validation performance reached before the maximum training duration.

Evaluation	Threshold	Accuracy	Precision	Recall	F1	ROC-AUC
Validation	0.50	90.77%	78.79%	52.00%	62.65%	90.85%
Validation	0.32	90.18%	64.91%	74.00%	69.16%	90.85%
Final Test	0.32	88.17%	60.61%	59.70%	60.15%	88.38%

A validation threshold of 0.32 improved recall to 74.00% on the validation set. This threshold was then evaluated once on the untouched test set, where the ANN achieved 88.17% accuracy, 60.61% precision, 59.70% recall, 60.15% F1 score, and 88.38% ROC-AUC. LightGBM remained preferable for production because it provided the stronger test ROC-AUC and a simpler tabular-data deployment path.

9.2 Autoencoder Anomaly Detection

An autoencoder was trained as an unsupervised anomaly-detection experiment using ten behavioural features: Total Spending, the six product-category spending variables, and Web, Catalog, and Store purchase counts. Scaling was fitted on the training partition only.

The autoencoder architecture was 10-8-4-8-10 and was trained using the Adam optimizer with mean squared reconstruction error. The anomaly threshold was defined as the 95th percentile of training reconstruction error, approximately 0.9463.

Partition	Anomalies	Records	Anomaly Rate
Training	90	1,789	5.03%
Test	26	448	5.80%

Because the dataset contains no ground-truth anomaly labels, these results represent records with unusually high reconstruction error rather than confirmed fraudulent, erroneous, or otherwise abnormal customers.

9.3 LSTM Feasibility Assessment

An LSTM was considered for modelling sequential customer purchase behaviour. However, the available dataset contains one aggregated row per customer rather than a sequence of timestamped purchase events. Dt_Customer represents customer enrolment and does not provide a purchase-event history.

Creating artificial purchase sequences from aggregated columns would introduce fabricated temporal information and produce a methodologically invalid experiment. Therefore, an LSTM model was deliberately not trained. A valid future LSTM implementation would require transaction-level data containing at minimum a customer identifier, transaction timestamp, and purchase value or product/category information.

This is a documented dataset limitation rather than an omitted experiment. The project prioritizes valid modelling assumptions over artificially satisfying a model-count requirement with synthetic sequences.

10. Production Architecture

The project was structured as an end-to-end machine learning engineering system rather than only a modelling notebook. Reusable modules handle data processing, prediction, customer segmentation, explainability, configuration, database access, and logging. FastAPI exposes prediction services, Streamlit provides the user interface, PostgreSQL stores results, and Docker Compose orchestrates the complete application stack.

Customer Behavior Prediction - System Architecture

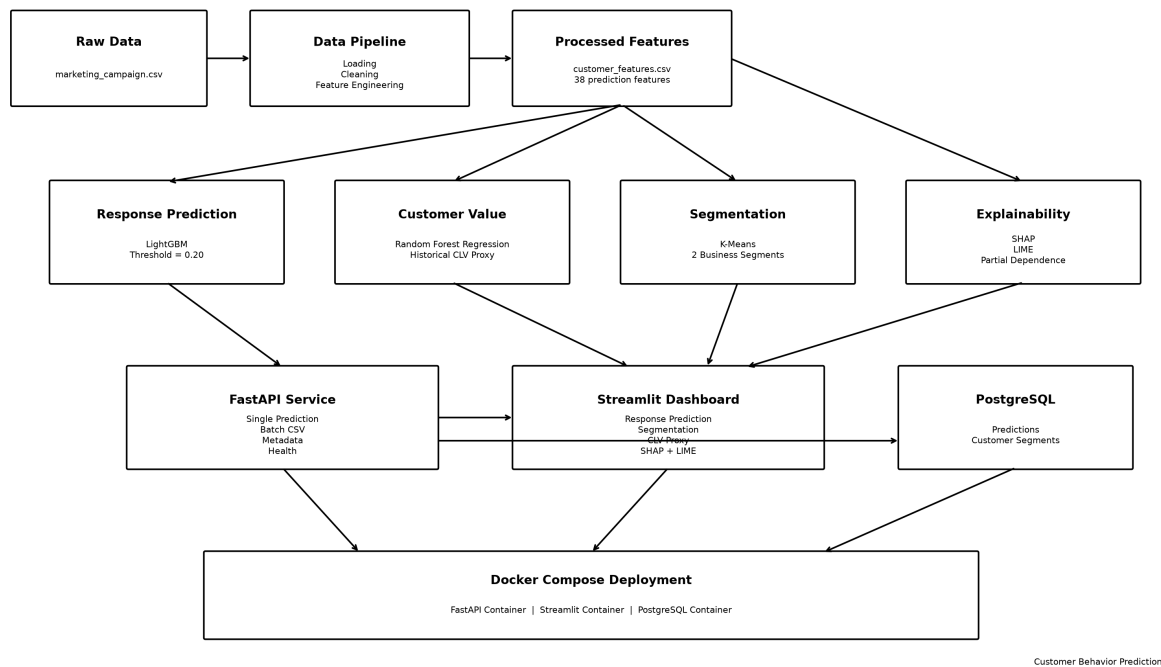


Figure 1. End-to-end architecture of the customer behaviour prediction system.

11. FastAPI Prediction Service

The production API is implemented using FastAPI with Pydantic request validation. The service provides health checking, model metadata, single-customer response prediction, and batch CSV prediction capabilities.

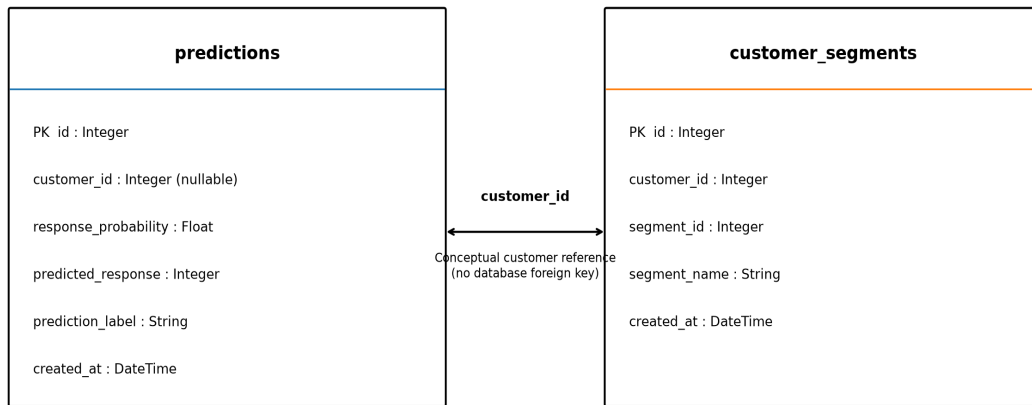
Capability	Purpose
Health	Confirms that the prediction service is running.
Metadata	Exposes model and deployment metadata.
Single Prediction	Accepts one validated customer profile and returns response probability, class prediction, label, and threshold.
Batch Prediction	Accepts a CSV file and returns predictions for multiple records.

The single-prediction endpoint was validated through the Dockerized Swagger interface, and prediction results were successfully persisted to PostgreSQL. The batch CSV workflow was also tested successfully.

12. PostgreSQL Persistence

SQLAlchemy is used to integrate the application with PostgreSQL. The database contains separate persistence structures for model predictions and customer-segment assignments. Database tables are initialized automatically when the API container starts.

Customer Behavior Prediction - Database ER Diagram



PostgreSQL persistence layer

Prediction results and customer segment assignments are stored independently.

Figure 2. Database entity-relationship view. The customer_id relationship shown between prediction and segmentation data is conceptual; no database foreign-key constraint is imposed.

Database persistence was verified directly in the running PostgreSQL container for both response predictions and customer segment assignments.

13. Streamlit Dashboard

The Streamlit dashboard provides an interactive interface for customer-level analysis. Users can enter key customer attributes and obtain the campaign-response probability and label, customer segment, historical CLV proxy estimate, SHAP explanation, and LIME explanation in a single workflow.

To keep the demonstration interface manageable, seven important customer inputs are directly editable: Income, Age, Recency, Total Spending, Web Purchases, Catalog Purchases, and Store Purchases. Remaining production-model inputs are populated from median values in the processed dataset. Consequently, the dashboard is a project demonstration interface rather than a complete customer-data entry system.

14. Dockerized Deployment

Docker Compose orchestrates three application services: the FastAPI API, the Streamlit dashboard, and PostgreSQL. Environment variables are used for database configuration so credentials and deployment-specific connection details do not need to be embedded directly in application code.

Service	Role	Port
API	FastAPI prediction and metadata service	8000
Dashboard	Streamlit customer analytics interface	8501
Database	PostgreSQL persistence	5432

The complete stack was successfully built and executed with Docker Compose. PostgreSQL health checking, automatic database initialization, API startup, Swagger prediction, database persistence, and Streamlit dashboard access were all verified.

15. Testing, Quality, and Reproducibility

Automated tests cover feature engineering, prediction logic, API behaviour, segmentation, customer-value prediction, configuration handling, database-related functionality, and explicit leakage checks. The final verified test suite contains 26 passing tests with 76% coverage across src and api.

Ruff static analysis completes with zero reported errors. Reusable functions include type hints and documentation where appropriate, and structured JSON logging is implemented for production prediction activity without logging the complete customer payload.

Reproducibility is supported through a fixed random seed of 42, YAML-based configuration, saved model artifacts and feature configurations, pinned scikit-learn version, environment-variable based deployment settings, and a one-command raw-to-processed data pipeline.

Quality Check	Verified Result
Automated tests	26 passed
Coverage	76%
Leakage tests	Passed
Ruff	Zero errors
Raw-to-processed pipeline	Verified
Docker Compose stack	Verified
API prediction	Verified
PostgreSQL persistence	Verified
Streamlit dashboard	Verified

16. Project Limitations

- The dataset is relatively small and represents aggregated customer behaviour rather than a live transactional system.
- The customer-value target is historical Total Spending and is not a true future Customer Lifetime Value target.
- A valid LSTM experiment cannot be performed because transaction-level time-ordered purchase sequences are unavailable.
- The autoencoder has no ground-truth anomaly labels, so detected records are candidates for investigation rather than confirmed anomalies.
- The dashboard exposes seven primary editable fields and fills the remaining model features using processed-data medians.
- Observed relationships between customer segments, features, and response behaviour are associations and must not be interpreted as causal effects.

17. Business Recommendations

- Use the LightGBM response probability to prioritize campaign audiences rather than targeting all customers equally.
- Use the 0.20 operating threshold when increased responder recall is valuable, while reviewing the precision-recall trade-off when campaign costs or business objectives change.
- Treat the higher-value K-Means segment as a useful group for retention, loyalty, and premium campaign analysis, while validating campaign strategies through controlled experiments.

- Use SHAP and LIME explanations when reviewing individual predictions so that marketing decisions are not based solely on an unexplained score.
- Collect transaction-level timestamped purchase data in future systems to enable genuine sequential modelling and future-value forecasting.

18. Conclusion

The project demonstrates a complete machine learning engineering workflow for customer behaviour analysis. It progresses from raw data validation and feature engineering through classical machine learning, customer segmentation, customer-value estimation, explainability, and deep-learning experimentation to a deployable FastAPI, Streamlit, PostgreSQL, and Docker Compose application.

LightGBM was selected as the production response model based on its strong ROC-AUC and useful recall after threshold tuning. K-Means provides interpretable customer segments, while Random Forest regression provides a strong historical customer-value proxy. SHAP and LIME improve transparency around individual predictions.

Where the available data could not support a valid modelling requirement, particularly sequential LSTM modelling and true future CLV prediction, the limitation was documented explicitly rather than introducing fabricated data or overstating the capabilities of the system.

Notebook Organization Note

The experimental work is retained in a single comprehensive notebook, `notebooks/01_eda.ipynb`, rather than being physically split into separate EDA, feature-engineering, and model-experiment notebooks. The notebook contains the complete experimental workflow, while production functionality has been separated into reusable modules under `src`, `api`, and `frontend`.